Chennai Enquiry Bot using Retrieval-Augmented Generation (RAG) and Local LLM (Ollama)

Arjun G Sundar – 2024179001 MCA SS

Part A: Design of the Chatbot

1. Domain

The chosen domain is a **Tourist & Local Enquiry Bot for Chennai**.
The chatbot is designed to help:

- **Tourists** visiting Chennai → who want to know about popular places, local food, transport, weather, and cultural experiences.
- **Locals** in Chennai → who may want quick info on shopping centers, entertainment, or latest events.

This domain was selected because:

- Chennai is a **major metro city** with rich cultural and tourist attractions.
- Both **visitors and residents** require information quickly.
- A chatbot provides **instant, conversational access** without browsing multiple websites.

2. Intents (User Goals) and Entities (Important Keywords)

Intent	Example Queries	Entities (Keywords)
Greeting	Hi, Hello, Hey	"hi", "hello", "hey"
Tourist places	Tell me about places, Where should I visit in Chennai?	Marina Beach, Kapaleeshwarar Temple, Fort St. George
Food	What food is Chennai famous for? Best dishes in Chennai?	dosa, idli, filter coffee, Chettinad chicken, biryani
Shopping	Where can I shop in Chennai?	T. Nagar, Express Avenue, Phoenix Mall, Pondy Bazaar
Movies & Entertainment	Which theatres are best? Any movie halls nearby?	Sathyam Cinemas, PVR, multiplex
Transport	How do I travel in Chennai?	metro, bus, suburban train, auto
Weather / Best time to visit	What's the weather like? When should I visit Chennai?	hot, monsoon, November–February

Intent	Example Queries	Entities (Keywords)
Culture & Festivals	Tell me about Chennai's culture. Any festivals?	Bharatanatyam, Carnatic music, Pongal, Music Season
Personalization	My name is Arjun. What's my name?	name recognition
Exit	Bye, Exit, Quit	"bye", "exit", "quit"

3. Conversation Flow Design

The chatbot follows a **retrieval + generation pipeline**:

1. **User Input** → The user types a query (e.g., *"Where can I shop in Chennai?"*).
2. **Retriever** → Uses semantic search (embeddings) to find the most relevant info from the knowledge base.
3. **Generator (LLM)** → Takes the retrieved info + user query and generates a natural response.
4. **Output** → User receives a clear, polite, and conversational answer.
5. **Fallback** → If the query doesn't match, chatbot suggests available topics (places, food, shopping, movies).
6. **Personalization** → If the user provides their name, the chatbot remembers and personalizes greetings.

4. Conversation Rules

1. **IF** user says "Hi" or "Hello"
   THEN chatbot replies "Hello! I'm your Chennai Enquiry Bot. Ask me about places, food, shopping, movies, or culture."
2. **IF** user asks about tourist places
   THEN chatbot replies with "Some top attractions in Chennai are Marina Beach, Kapaleeshwarar Temple, Santhome Cathedral, and Fort St. George. You may also visit Mahabalipuram nearby."
3. **IF** user asks about food
   THEN chatbot replies with "Chennai is famous for dosa, idli, authentic filter coffee, Chettinad chicken, and Chennai-style biryani."
4. **IF** user asks about shopping
   THEN chatbot replies with "For shopping, visit T. Nagar for sarees, Pondy Bazaar for street shopping, and malls like Express Avenue and Phoenix MarketCity."

5. **IF** user asks about movies
   THEN chatbot replies with “Sathyam Cinemas and PVR are top theatres to watch Tamil and international movies.”
6. **IF** user asks about transport
   THEN chatbot replies with “You can use metro, MTC buses, suburban trains, or autos to travel around Chennai.”
7. **IF** user asks about weather or best time to visit
   THEN chatbot replies with “Chennai is hot most of the year, but November to February is the most pleasant time to visit.”
8. **IF** user asks about culture
   THEN chatbot replies with “Chennai is known for Bharatanatyam dance, Carnatic music, Pongal festival, and the December Music Season.”
9. **IF** user says “My name is Arjun”
   THEN chatbot replies “Nice to meet you, Arjun! I’ll remember your name.”
10. **IF** user query is not understood
    THEN chatbot replies “Sorry, I didn’t get that. Do you want info on places, food, shopping, or movies?”

5. Justification of Design

- The **domain** ensures the chatbot has a practical use for both tourists and locals.
- **Intents and entities** cover the most common topics people would ask about Chennai.
- The **conversation rules** ensure structured and consistent responses.
- By using **RAG + LLM**, the chatbot is flexible and can handle variations in user queries, unlike rigid rule-based bots.

Part B: Implementation

chennai_knowledge.json

```
{
   "places": "Top tourist places in Chennai include Marina Beach, Kapaleeshwarar Temple, Santhome Cathedral, Fort St. George, and Mahabalipuram nearby.",
   "food": "Famous foods in Chennai are dosa, idli, filter coffee, Chettinad chicken, and Chennai-style biryani.",
   "shopping": "For shopping, visit T. Nagar for sarees and jewelry, Pondy Bazaar for street shopping, and Express Avenue or Phoenix Mall for brands.",
```

"movies": "Sathyam Cinemas and PVR in Phoenix Mall are the best theatres to watch Tamil and international movies.",

"transport": "You can travel around Chennai by metro, MTC buses, suburban trains, and autos.",

"weather": "Chennai is hot most of the year. The best time to visit is November to February when the weather is cooler.",

"culture": "Chennai is known for Bharatanatyam dance, Carnatic music, and cultural festivals like Pongal and the December Music Season."

}

chennai_bot.py

```
import json

import numpy as np

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity

import ollama


with open("chennai_knowledge.json", "r") as file:

    documents = json.load(file)


doc_texts = list(documents.values())

doc_keys = list(documents.keys())


embed_model = SentenceTransformer("all-MiniLM-L6-v2")

doc_embeddings = embed_model.encode(doc_texts)


def retrieve_context(query, top_k=2):

    query_embedding = embed_model.encode([query])

    similarities = cosine_similarity(query_embedding, doc_embeddings)[0]

    top_indices = np.argsort(similarities)[::-1][:top_k]
```

```
    return [doc_texts[i] for i in top_indices]

def rag_llm_chatbot(query, user_name=None):
    context = retrieve_context(query)
    prompt = f"""
    You are a Chennai Enquiry Bot for tourists and locals.
    Use the following context to answer naturally:

    Context: {" ".join(context)}
    User Query: {query}
    """
    response = ollama.chat(
        model="mistral",
        messages=[
            {"role": "system", "content": "You are a helpful Chennai tourist assistant."},
            {"role": "user", "content": prompt}
        ]
    )
    answer = response['message']['content']
    if user_name and "hi" in query.lower():
        answer += f" Nice to see you again, {user_name}!"
    return answer

def chat():
    print("CHENNAI Enquiry Bot: Hello! Ask me about places, food, shopping, movies, or culture.")
    user_name = None
    while True:
        query = input("You: ")
```

```
        if "bye" in query.lower():
            print("Bot: Goodbye! Have a great time in Chennai ")
            break
        elif "my name is" in query.lower():
            user_name = query.split("is")[-1].strip().capitalize()
            print(f"Bot: Nice to meet you, {user_name}! I'll remember your name.")
            continue
        elif "what's my name" in query.lower():
            if user_name:
                print(f"Bot: Your name is {user_name}.")
            else:
                print("Bot: I don't know your name yet. Tell me using 'My name is ...'")
            continue
        response = rag_llm_chatbot(query, user_name)
        print("Bot:", response)

if __name__ == "__main__":
    chat()
```

Part C: Extension

The chatbot was extended beyond basic rule-based functionality by adding a **knowledge base file** and a **learning component**.

1. Knowledge Base File (External JSON Storage)

Instead of hardcoding responses inside Python if-else rules, the chatbot uses an **external JSON file** (chennai_knowledge.json) to store information.

Advantages of this design

- **Modular** → Easy to update or expand by editing the JSON file.
- **Scalable** → More categories (like nightlife, IT hubs, healthcare, nearby trips) can be added later.

- **Separation of concerns** → The chatbot logic is in Python, while content is stored outside.

Structure of JSON File

The JSON file stores **key–value pairs**, where the **key** is the category (intent), and the **value** is the corresponding response text.

How it works in the chatbot:

1. The chatbot loads the JSON file once at the start.
2. When a user asks a query, the retriever matches it with the most relevant JSON entries using **embeddings + cosine similarity**.
3. The matched context is passed to the LLM (Ollama Mistral) to generate a natural answer.

This makes the bot **dynamic** and avoids manually writing long if-else conditions.

2. Learning Component (Memory Feature)

The chatbot was enhanced with a **personalization feature**: it remembers the **user's name**.

How it works

- If the user types:
- My name is Arjun

The chatbot extracts "Arjun", stores it in a variable (user_name), and acknowledges with:

Nice to meet you, Arjun! I'll remember your name.

- Later, if the user asks:
- What's my name?

The chatbot recalls the stored name and responds:

Your name is Arjun.

- If the user hasn't introduced themselves yet, the chatbot politely says:
- I don't know your name yet. Tell me using 'My name is ...'

Benefits of Memory Feature

- Adds **human-like interaction** → the bot feels more personal.
- Improves **user engagement** by remembering past details.

- Can be extended to remember preferences (e.g., favorite food, preferred shopping places).

3. Additional Fallback Handling

The chatbot was extended with **fallback responses** to ensure smooth conversation even when user queries don't match the knowledge base.

Examples:

- If the user asks:
- What is the capital of Tamil Nadu?

and it's not in the JSON, the bot replies:

Sorry, I didn't get that. Do you want info on places, food, shopping, or movies?

- This ensures that the bot never "goes silent" and always guides the user back to supported topics.

4. Summary of Extensions

- **Knowledge Base File** → stores information externally in JSON, making the bot modular and scalable.
- **Learning Component (Memory)** → bot remembers user's name and responds personally.
- **Fallback Handling** → bot gracefully handles unrecognized queries.